

Scalable Notification System — Performance Experiments Report

1. System under test, instrumentation, and validity boundaries

The system under test is an **asynchronous notification pipeline**, not a synchronous end-to-end delivery API. The ingress service accepts **POST /notifications** and, in one database transaction, persists both the notification row and a transactional outbox row for **NotificationRequested**. This means client-visible accept latency is inherently sensitive to the synchronous **RDS write path**, connection-pool pressure, and application thread contention before Kafka or downstream workers even become relevant.

Downstream, the design deliberately separates **EMAIL** and **INAPP** work into distinct topics and ECS services. That choice is what makes the experiments interpretable: worker scale-out can be changed manually per channel, burst behavior can be observed without a policy engine hiding the bottleneck, and failure isolation can be evaluated under controlled provider slowdown.

Aspect	What was measured / why it matters	Important caveat
Workload	Headless Locust against the public ALB endpoint; EMAIL-only or MIXED traffic; fixed 256 B payload	No read-heavy traffic, auth, or large-payload sweep
Kafka lag	Consumer-group lag sampled on the Kafka EC2 host to detect durable backlog	Lag = 0 can mean "no backlog" or simply that queueing never became the dominant visible bottleneck
CloudWatch	ECS CPU / memory for ingress and workers; max values used heavily for interpretation	Downsampled windows can smooth short spikes; short-lived saturation may be underrepresented
Attempts API	Intended to validate attempt-level and end-to-end timing claims	Recorded AWS runs returned no usable attempt samples, so worker timing claims remain indirect

Why these experiments still matter without auto scaling. The study intentionally held capacity **fixed** and changed it **manually** across runs. That is a feature, not a weakness: it isolates the architecture and reveals where the next scaling dollar should be spent. Auto scaling only makes sense after the system's throughput, tail-latency, and backlog behavior are already understood under fixed capacities.

Experiment	Purpose	Primary tradeoff
A. Worker scale-out	Measure how EMAIL worker replicas change throughput and tail latency at fixed offered load	More replicas vs. higher cost and diminishing returns
B. Burst buffering	Observe system behavior during a baseline → spike → recovery workload	Fast acceptance vs. ingress/DB saturation under bursts
C. Failure isolation	Check whether slow email processing harms in-app behavior under mixed traffic	Isolation benefits vs. more services/topics and operational complexity

2. Experiment A — Fixed EMAIL load vs. worker replicas

Purpose	Measure horizontal scalability of the EMAIL worker tier under a fixed workload.
Tradeoff explored	Additional replicas may reduce tail latency and increase sustained throughput, but they also increase cost and coordination overhead.
Key limitation	Kafka lag stayed at 0 in all recorded runs, so the clearest bottleneck signal came from ingress CPU and API latency rather than queue growth.

Configuration. EMAIL-only traffic, 80 Locust users, 300 s run, 256-byte payload, replicas = 1/2/4/8. Results are from `expA_email_fixedload_ec2_20260421_141019`.

Replicas	Req/s	p95 ms	p99 ms	Avg ms	Ingress CPU max %	Email CPU max %	Lag max
1	293.4	480	1200	237.2	99.2	99.6	0
2	321.7	470	1100	214.4	93.4	1.4	0
4	434.8	260	380	150.4	99.5	89.1	0
8	446.3	250	350	145.4	99.6	99.5	0

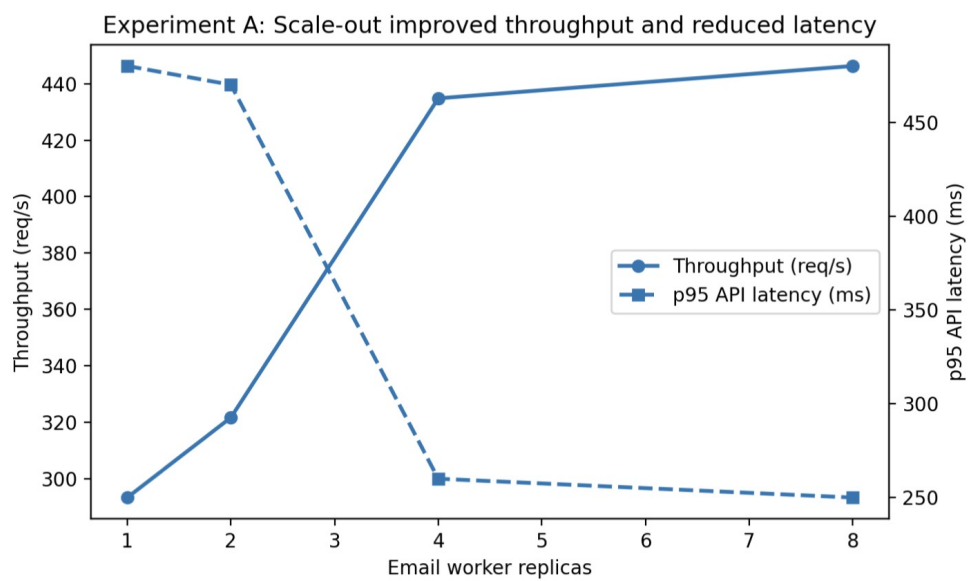


Figure A1. Throughput and API p95 latency both improve sharply up to 4 replicas, then flatten.

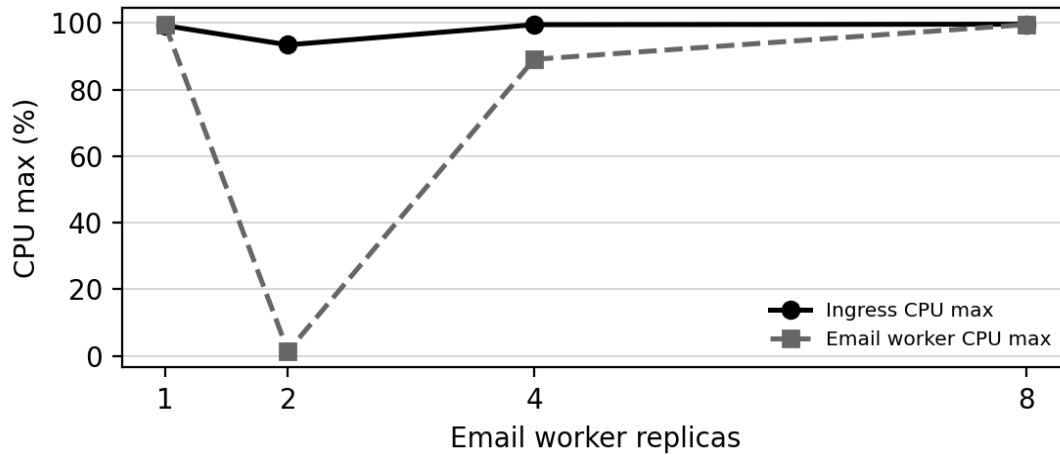


Figure A2. Ingress CPU remains near saturation across all runs, while email-worker CPU varies with available parallelism.

Analysis. The strongest return appears in the **2 → 4 replica** step: throughput rises from 321.7 to 434.8 req/s while p95 falls from 470 to 260 ms. The 4 → 8 step produces only a small extra gain (~2.6% throughput and 10 ms p95 improvement), which strongly suggests a practical **cost/performance knee at four EMAIL workers** for this offered load.

The important negative result is just as valuable as the positive one: **Kafka lag stayed at 0** across all runs. That means the worker tier was not the first place durable backlog appeared. Instead, the clearest stress signal is that **ingress CPU remained very high** in every run, reaching roughly 93–100% max. Together, those signals are consistent with an upstream bottleneck—most plausibly the synchronous ingress + DB accept path—rather than the email consumers themselves.

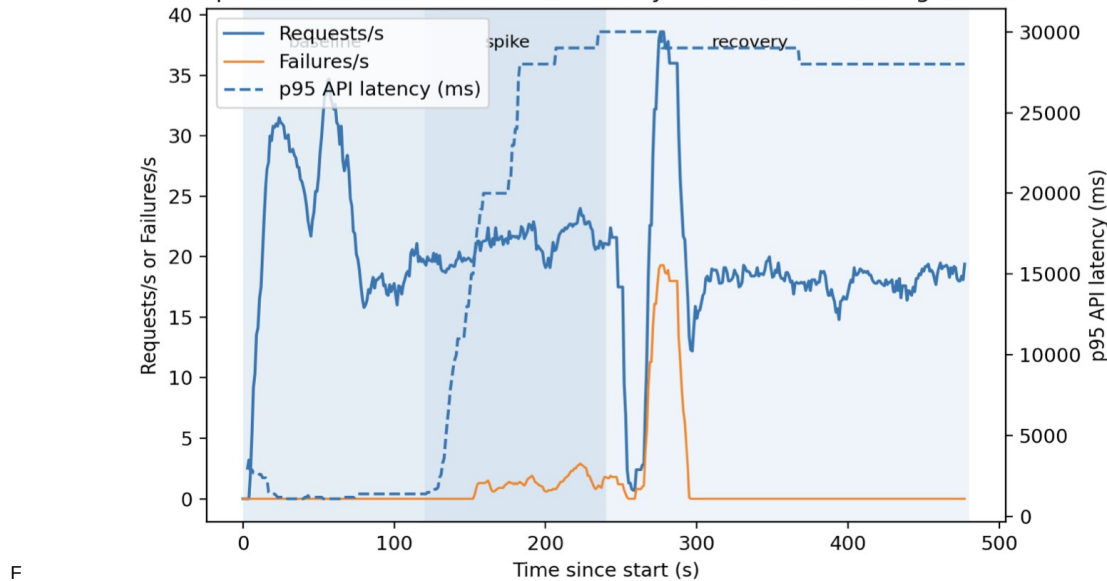
Interpretation for system design. Experiment A therefore does not say “more workers always help.” It says workers help until the system runs into a different bound. In this topology, once worker count reaches four, the dominant limit shifts toward ingress and RDS. That is exactly the kind of evidence needed before designing a sensible scaling policy.

Limitations. We did not independently sweep ingress task size, ingress count, DB pool size, partition count, or payload size. The result should therefore be read as a careful characterization of one AWS deployment rather than a universal optimum.

3. Experiment B — Burst traffic buffering and recovery

Purpose	Stress the system with baseline → spike → recovery load and observe API behavior, Kafka lag, and ECS resource signals.								
Tradeoff explored	Bigger spikes mimic flash crowds better, but may saturate the synchronous accept path before backlog is visible in Kafka.								
Key limitation	The two recorded runs use different severities, and Kafka lag stayed at 0 in both summaries; recovery therefore had to be interpreted mainly from API-side evidence.								
Run	Workers	Spike users	Success	Req/s	p95 ms	p99 ms	Ingress CPU max %	Email CPU max %	Lag max
B1 moderate	4	120	100%	25.58	4900	5700	11.27	2.43	0
B2 extreme	1	400	94.36%	20.28	28000	39000	99.64	1.99	0

Experiment B: burst traffic drove latency and failures at the ingress tier



Analysis. The moderate run (B1) completed with no HTTP failures, but already showed multi-second API tails. The extreme run (B2) pushed p95 to 28 s, p99 to 39 s, and still showed **email worker CPU below 2%** with **lag = 0**. The most plausible interpretation is that overload manifested **before Kafka became the durable queueing witness**—most likely in the ingress/DB accept path that must synchronously write both the notification row and outbox row before returning 202 Accepted.

This is an important systems point. Kafka lag is only informative when Kafka is where work accumulates. In these burst runs, the system appears to have degraded earlier, at the front door, so the broker never developed a visible durable backlog. That is why “lag stayed zero” should not be misread as “the system was healthy.” User-visible failure and tail latency were already severe.

What the results reveal about buffering. The burst experiment therefore distinguishes two types of resilience: **queue buffering resilience** and **accept-path resilience**. The system likely still had downstream queueing capacity available, but the ingress/DB layer could not transform incoming HTTP requests into durable queued work fast enough under the extreme spike.

Limitations. Without distributed tracing or direct DB instrumentation, the experiment cannot assign blame precisely among RDS write latency, connection-pool starvation, application thread exhaustion, or ALB timeout interactions. The conclusion is strong evidence rather than formal proof.

4. Experiment C — Mixed-channel failure isolation

Purpose	Validate whether slow EMAIL processing harms the INAPP path when channels use separate topics and separate ECS services.
Tradeoff explored	Per-channel isolation reduces blast radius, but costs more services, topics, and operational surface area.
Key limitation	Attempt-level samples were missing in all recorded rounds, so worker-side queue wait vs. processing breakdown could not be validated from data.

Round	Email lat. ms	In-app lat. ms	Req/s	Success	p95 / p99 ms	Email lag	In-app lag	In-app CPU max %	Ingress CPU max %
1 normal	100	20	26.76	100%	2700 / 2900	0	0	1.79	53.51
2 slow	1000	20	27.53	100%	2800 / 3100	0	0	1.79	23.81
3 v. slow	3000	20	27.71	100%	2800 / 3100	0	0	1.38	16.22

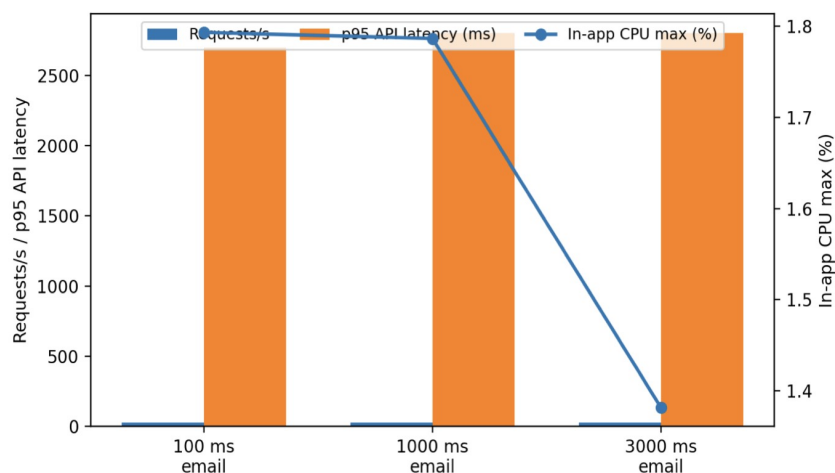


Figure C1. As email latency increases, requests/s and API p95 remain approximately flat while in-app CPU remains low.

Analysis. As email latency increased from 100 ms to 3000 ms, **requests/s stayed flat**, API p95 remained near 2.7–2.8 s, and **in-app CPU stayed around 1–2%**. That is the strongest evidence in the dataset for isolation: slow email work did not visibly steal in-app compute or destabilize HTTP acceptance under this mixed steady load.

What the results do **not** establish is queueing isolation under saturation. Both consumer-group lags stayed at 0 and the attempt-sampling path returned no usable samples. So the strongest supported claim is **service-level isolation**, not backlog divergence by channel. In other words, the architecture clearly prevented slow email work from consuming in-app worker compute, but this load level was not high enough to force visible durable backlog on the slow path.

Design implication. This matters because it validates the architectural choice of separate topics and separate worker services. Even before introducing circuit breakers or channel-specific autoscaling, the system already narrows blast radius by decoupling worker capacity domains.

Limitations. To force stronger lag evidence, a future run should raise offered load, lower email parallelism, or reduce partitioning so that the slow email path can accumulate visible durable backlog.

5. Cross-experiment synthesis and conclusions

Main findings. Experiment A shows that worker scale-out helps materially until the system hits an upstream bottleneck. Experiment B shows that burst overload becomes user-visible before Kafka lag necessarily grows. Experiment C shows that separate channel workers protect compute isolation even when one downstream path becomes slow. Taken together, the results paint a coherent picture: the architecture is directionally sound, but the **next scaling priority is the accept path**, not simply adding more downstream workers.

What these results imply for future autoscaling. Auto scaling would be useful, but only after the correct bottleneck is identified. The data here suggest that a naive worker-only policy would be incomplete: under harsh bursts, the system was more constrained by ingress and DB-side work than by email consumer utilization. A sensible next step would therefore combine modest worker scaling with a separate study of ingress task count/size and database pool tuning.

Threats to validity. The study used one AWS lab topology, one self-hosted single-broker Kafka deployment on EC2, fixed payload size, and mostly write-heavy traffic. CloudWatch metrics were sampled rather than continuously traced. Attempt-level timing was not available in the recorded AWS runs. These limitations do not invalidate the experiments, but they bound the strength of the claims: the report is best read as a careful characterization of this system under controlled load, not as a universal statement about all notification architectures.

Question	Evidence-backed answer
Where is horizontal scalability strongest?	EMAIL workers benefit meaningfully up to 4 replicas; 8 replicas adds little for the tested fixed load.
Can Kafka absorb bursts cleanly in these runs?	Not as the first visible bottleneck. Severe overload showed up as HTTP failure/tail latency before durable lag growth.
Do per-channel workers help?	Yes for compute isolation: slow EMAIL did not materially raise INAPP CPU or HTTP p95 in the mixed-load runs.
What remains unresolved?	Attempt-level end-to-end timing and queue/process breakdown; lag-zero ambiguity; separate sweeps of ingress and DB configuration.

Overall limitation statement. These results are specific to one AWS lab topology: ALB fronting ECS ingress, RDS MySQL, and self-hosted single-broker Kafka on EC2. They are best read as a careful characterization of this system under controlled load, not as universal claims about all notification architectures.

6. Evidence-driven implications for next iterations

Where the next engineering effort should go. The experiments suggest three priorities in order. First, the ingress/DB accept path deserves deeper study because it dominated the harshest burst behavior and remained hot even after worker scaling improved Experiment A. Second, the worker layer already shows useful horizontal scalability, but only until the ingress layer becomes the tighter bound. Third, the mixed-channel architecture appears to buy real isolation, which means future optimizations can be targeted per channel rather than forcing a one-size-fits-all worker pool.

What these results say about possible auto-scaling policies. If auto scaling is added later, the experiments imply that a worker-only policy would be incomplete. EMAIL worker replicas matter under sustained load, but the burst results show that user-visible pain can emerge while worker CPU and lag remain low. That means any future policy should treat **ingress CPU / admission-path pressure** and **worker backlog / utilization** as separate control problems rather than assuming the same signal governs both.

Follow-up change	Why it is justified by current evidence	Expected payoff
Sweep ingress task count/size	Experiment A and B both point to ingress remaining hot even when worker scaling helps	Better API tails and fewer front-door failures under burst
Instrument DB pool / write latency	Current causal story strongly implicates the synchronous write path but cannot prove it	Stronger attribution between app, pool, and database bottlenecks
Repair attempts sampling	Experiment C could not validate queue wait vs. processing from recorded AWS data	True end-to-end and per-channel latency distributions
Re-run C at higher offered load	Current mixed-channel load was too gentle to force durable backlog divergence	Stronger evidence for or against queueing isolation

How to read the zero-lag results correctly. Across all three experiments, lag rarely became the decisive signal. That does *not* mean Kafka was unnecessary; it means the broker was not the first component to saturate in these runs. In Experiment A, the system was able to increase throughput by adding workers, but the combination of flat lag and persistently high ingress CPU suggests the new headroom was quickly consumed by the synchronous accept path. In Experiment B, overload surfaced as HTTP failures and large p95/p99 spikes before durable queueing growth could appear. In Experiment C, lag also stayed flat because the offered load was not high enough to make the slow EMAIL path accumulate visible backlog.

Why that distinction matters for the report. A common mistake in asynchronous-system analysis is to assume that “lag is the truth” for every overload episode. These experiments show a more nuanced pattern: sometimes the front door fails before the queue becomes the bottleneck. That is itself a valuable result, because it separates **accept-path resilience** from **drain-path resilience** and prevents the report from overclaiming that Kafka was or was not the limiting component.

Claim	Supported strength	Reason
Worker scaling improves throughput up to a point	Strong	Observed consistently in Experiment A with clear throughput/p95 improvement up to 4 replicas
Ingress/DB path is the next likely bottleneck	Moderately strong	Supported by high ingress CPU and severe burst tails, but not isolated with direct DB tracing
Per-channel workers provide compute isolation	Strong	INAPP CPU remained flat while EMAIL slowdown increased substantially
Per-channel workers provide queueing isolation under saturation	Weak / not established	Lag remained zero and attempts sampling was unusable in recorded runs

Final takeaway. The report therefore supports a disciplined interpretation: the architecture already demonstrates useful worker scalability and meaningful service-level isolation, but the most actionable next bottleneck is the synchronous ingress + outbox accept path. That conclusion is exactly the kind of result a five-page systems report should deliver: not just whether the system survived load, but **where the next engineering improvement is most likely to pay off**.

7. Reporting-ready interpretation of the tradeoffs

For the purposes of a course report, the cleanest way to summarize the tradeoffs is to separate them into three layers. **Capacity tradeoff:** Experiment A shows that buying more worker replicas improves sustained throughput and API tails, but only until another subsystem becomes dominant.

Admission tradeoff: Experiment B shows that the system can fail at the synchronous admission boundary before the asynchronous queue becomes the obvious witness. **Isolation tradeoff:** Experiment C shows that splitting channels into separate topics and services does buy protection, but the cost is additional deployment and operational surface area.

This layered interpretation matters because it avoids two common overstatements. First, it avoids claiming that worker scaling alone solves end-to-end scalability. Second, it avoids claiming that queue lag is always the primary evidence of overload. In this project, the strongest evidence often came from a combination of API tails, ingress CPU, and the *absence* of durable backlog growth where one might otherwise expect it.

Tradeoff category	What the current evidence supports	What remains uncertain
Horizontal scaling	Workers scale usefully up to a knee around 4 replicas in Experiment A	Whether the same knee holds under larger payloads or different partitioning
Burst resilience	Extreme bursts surface as front-door pain before Kafka backlog dominates	How much of that pain is DB-limited vs. app-thread or pool limited
Failure isolation	Separate worker services protect INAPP compute from slow EMAIL work	Whether the same architecture would also preserve low backlog under much heavier mixed traffic

In short, the report should be read as evidence that the architecture is **scalable in parts but not uniformly scalable everywhere**. The worker tier demonstrates usable horizontal elasticity under fixed load, while the ingress + database layer remains the more plausible next bottleneck under extreme offered load. That is a stronger and more honest conclusion than simply saying the system “scaled” or “did not scale.”

Connection back to the course goals. From a scalable distributed systems perspective, the value of these experiments is that they expose where asynchronous design helps and where it does not automatically remove bottlenecks. The outbox + Kafka design clearly decouples downstream delivery from the HTTP response path, which is why Experiment C can show strong service-level isolation without large changes in API p95. At the same time, Experiment B demonstrates that decoupling the back end does not magically make the front door unbounded; the synchronous accept path still has to durably record work before the system can benefit from asynchronous draining.